

RAG PROOF-OF-CONCEPT COMPARISON

POC Comparison Report

A structured comparison of RAG pipeline proofs-of-concept, built to be extended as more team members' POCs are added. This edition compares the Vector Technics POC against the Kunal/Rithish 3-strategy benchmark POC.

Living document — see "How to add a POC" on the next page. Source data: rag-strategy.md, eval_results.json, and the Kunal/Rithish POC write-up.

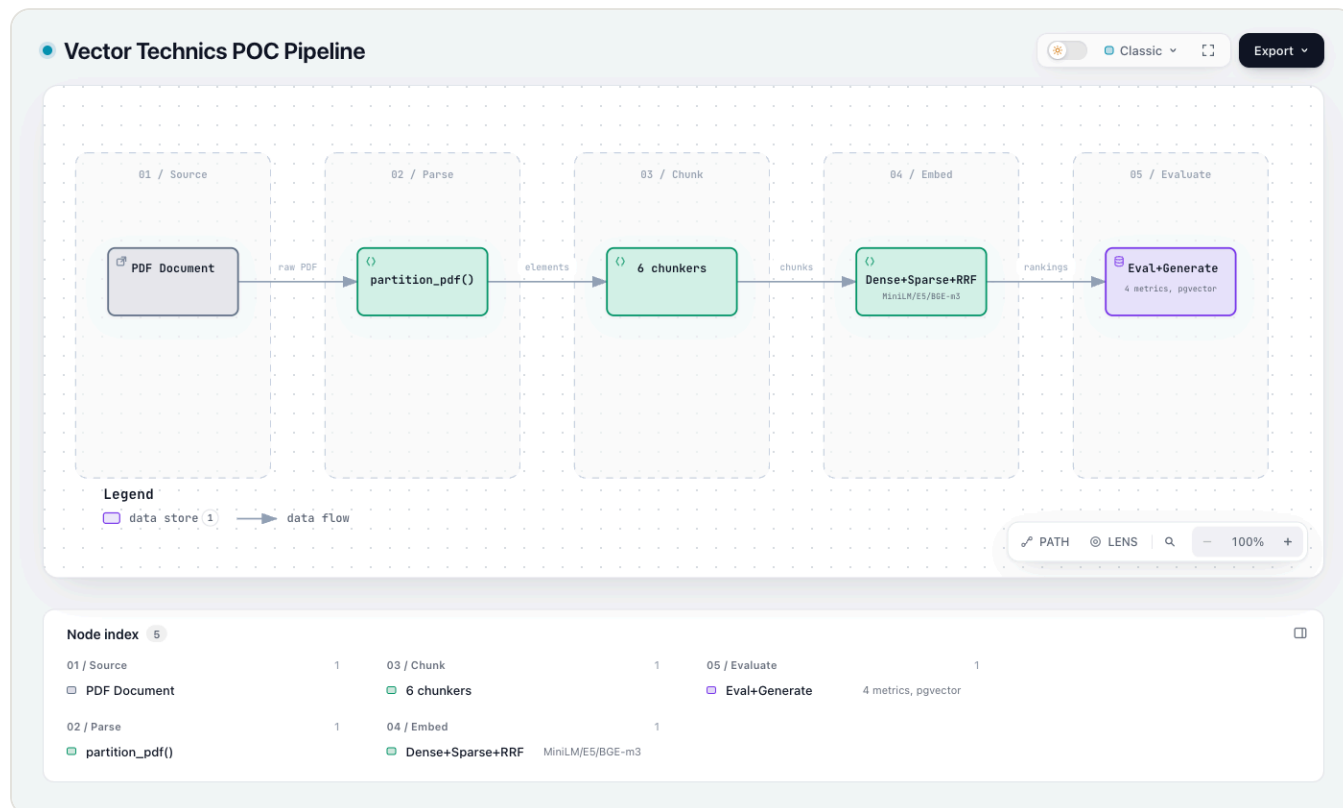
How to add a new POC

This report is designed to grow. When a new team member's POC comes in:

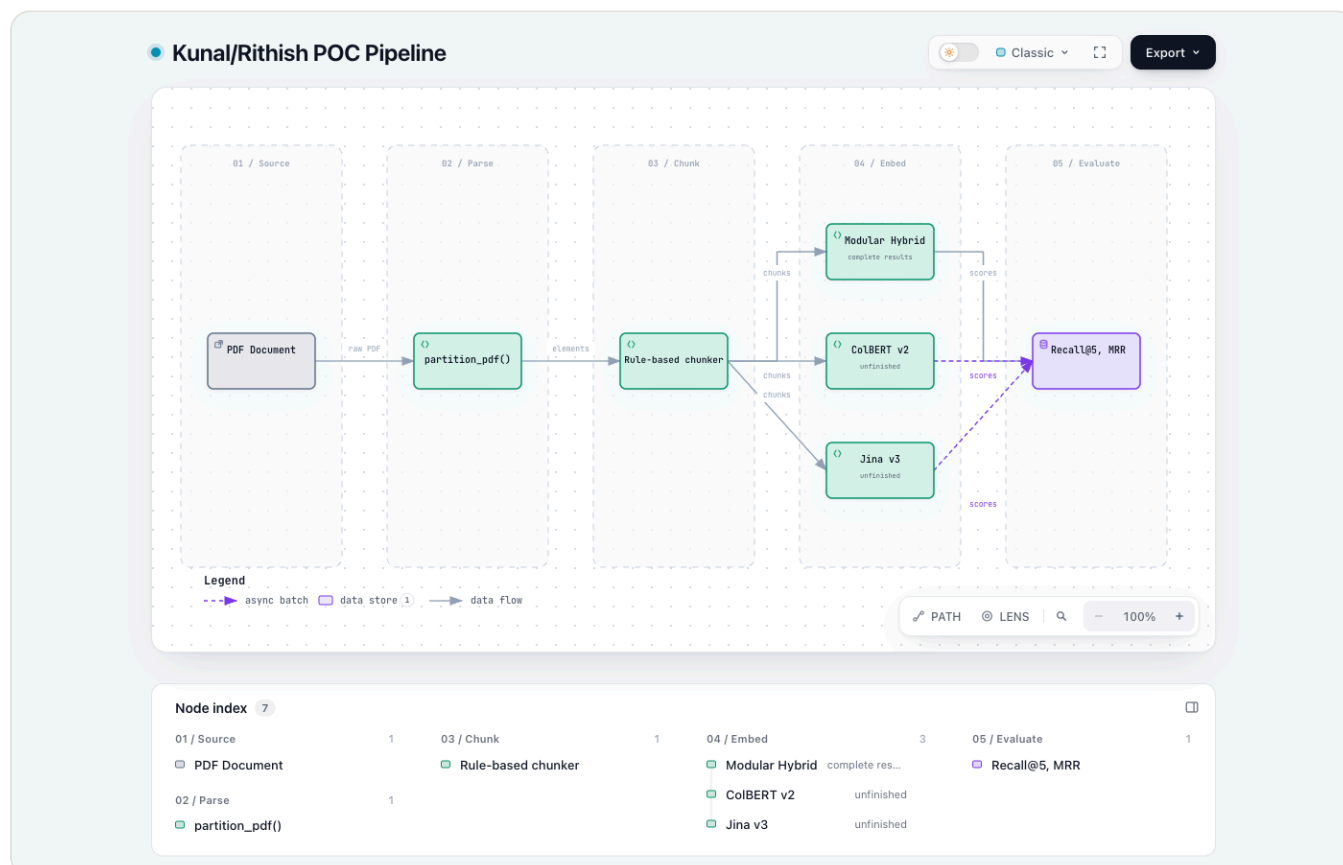
- Add a new column to the comparison table and a new pipeline diagram in the same style.
- Add its scores to the retrieval-quality and scope-breadth charts as a third bar/series.
- If it introduces a criterion the table doesn't have yet, add a row rather than force-fitting it.

Pipeline comparison

Both POCs share the same ingestion library but diverge from chunking onward.



OURS Vector Technics POC — 6 chunking strategies, dense+sparse embedding fused with Reciprocal Rank Fusion, full eval + generation + persistence.

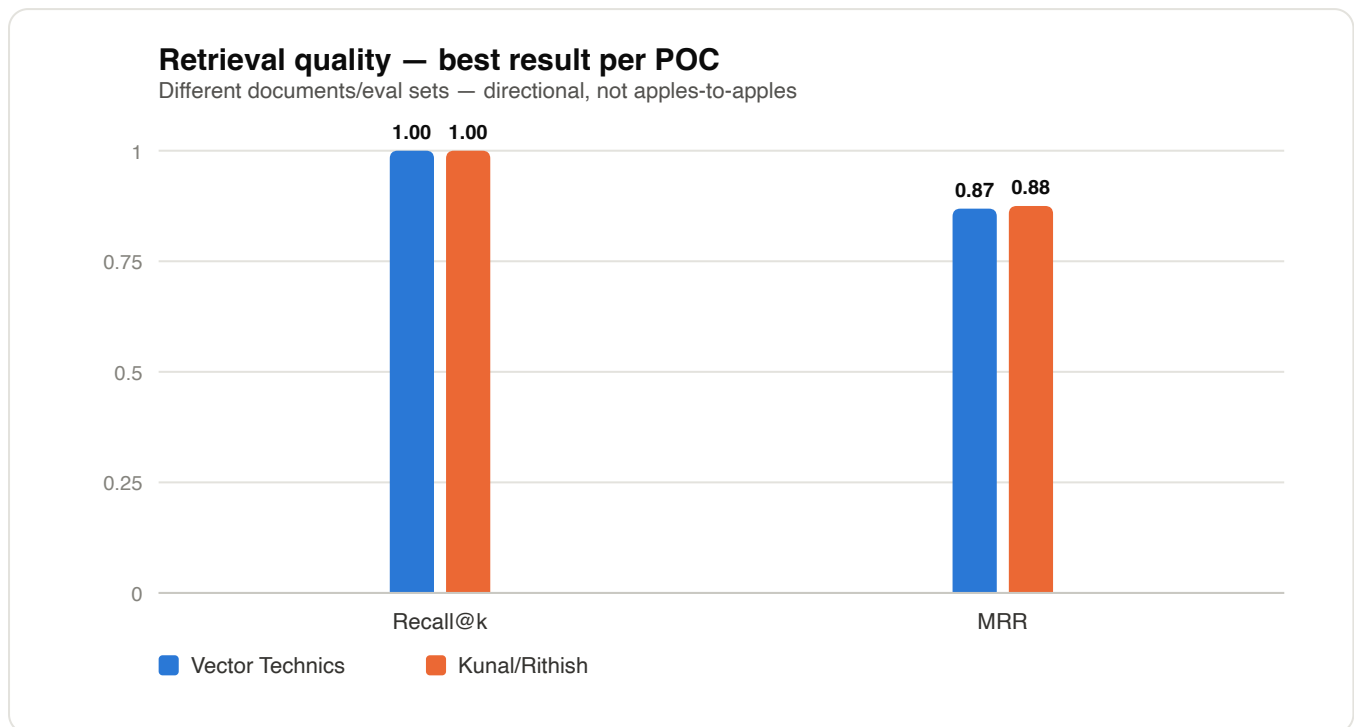


THEIRS Kunal/Rithish POC — 1 chunking strategy, 3 distinct embedding architectures benchmarked in parallel, only 1 of 3 finished scoring.

Comparison table

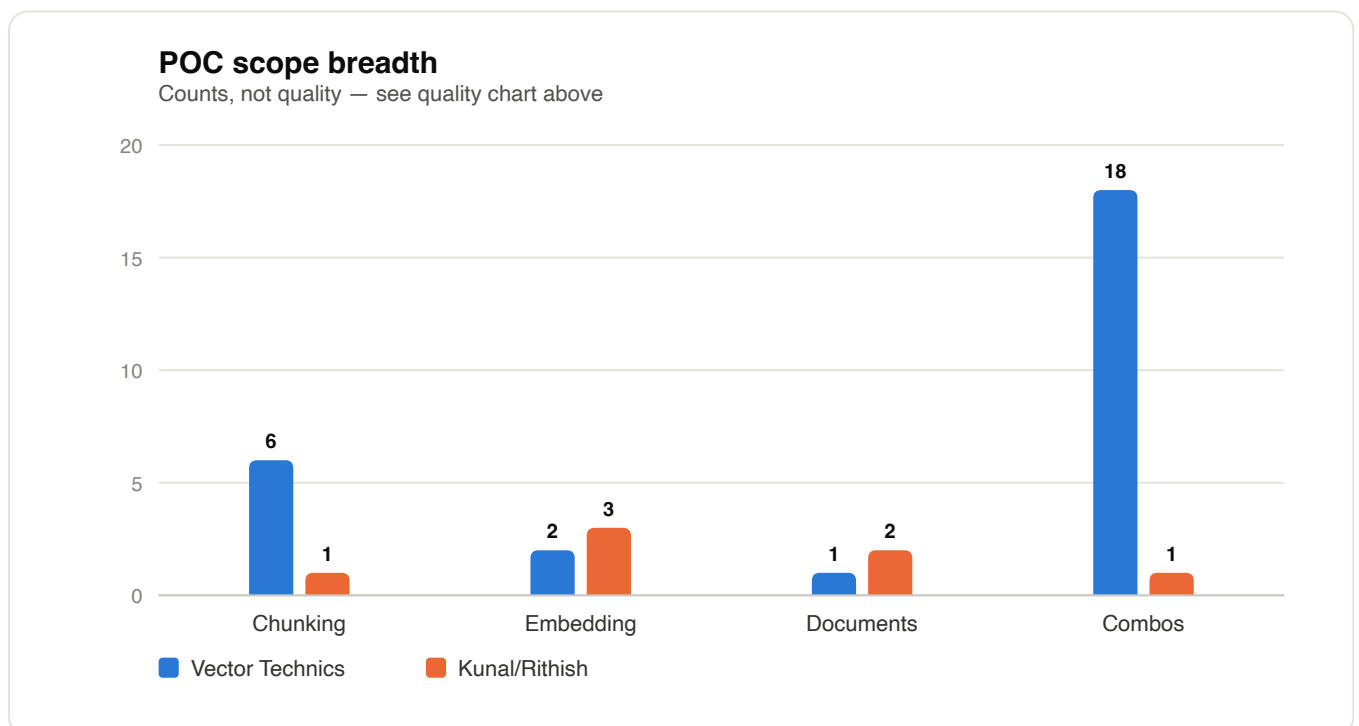
Criterion	Vector Technics (ours)	Kunal/Rithish (theirs)
Ingestion	<code>unstructured.partition_pdf()</code>	<code>unstructured.partition_pdf()</code>
Chunking strategies tried	6 (fixed, recursive, sentence, structure-aware, semantic, LLM-contextual)	1 (rule-based: recursive splitter + title boundaries + intact tables)
Embedding architectures	2 (dense, sparse)	3 (hybrid, ColBERT late-interaction, Jina asymmetric adapters)
Hybrid fusion method	Reciprocal Rank Fusion (rank-based)	alpha-weighted average, alpha=0.7, untuned
Eval metrics	recall@k, precision@k, MRR, NDCG, + ragas LLM-judged	Recall@5, MRR
Combos evaluated to completion	18+	1 (2 of 3 architectures unfinished)
Documents tested	1 (28 questions)	2 (13MB and 4KB documents)
Generation step	Yes — Claude Haiku, cited sources	Not present
Persistence	pgvector + CLI	In-memory only
Results viewer	Interactive dashboard	Static tables/chart

Retrieval quality — best result per POC



Ours: contextualized chunking + BGE-m3 hybrid, on a 28-question eval set. Theirs: Modular Hybrid on the harder Bharath Components document. Different documents and eval sets — read as directional, not a controlled A/B.

POC scope breadth



Breadth of what was tried, not quality of the result — pair with the chart above.

Similarities

- Same ingestion library and same base chunker, with a title/structure-aware variant in both.
- Both ground-truth sets use literal text matching (substring/keyword) — neither catches paraphrased answers.
- Both explicitly flag their eval sample sizes as too small for statistical significance, valid for correctness only.

Differences

Vector Technics strengths

- 6 chunking strategies swept vs. 1
- RRF fusion instead of an untuned weighted average
- Only POC with generation, persistence, and a dashboard

Kunal/Rithish strengths

- 3 distinct embedding architectures attempted, not just dense+sparse
- ColBERT and Jina-style adapters don't exist in our codebase at all
- Tested on 2 documents of very different size/difficulty vs. our 1

Which is better

If the goal is "which embedding architecture wins": their POC is more credible in intent (3 architectures) but incomplete in execution — only 1 of 3 has full benchmark numbers, so it doesn't yet answer its own question.

If the goal is "can we ship a working RAG system": ours is further along — the only one with a generation step, persistent storage, and a CLI someone could run against a new document today.

Recommendation: port their ColBERT/Jina embedding code into our pipeline (it already has the chunking sweep, eval harness, and RRF fusion to properly benchmark them) rather than porting our generation/persistence work into theirs — combine strengths instead of picking a winner.

Open questions for the team

- Do we need late-interaction (ColBERT) or asymmetric adapters (Jina), or is dense+sparse+RRF sufficient? Their POC doesn't answer this since ColBERT/Jina numbers were never completed.
- Should ground truth move to LLM-judged or human-verified, given both POCs share the substring/keyword-matching weakness?
- Is our sparse embedding step batched? Their unbatched SPLADE loop caused an 18-minute index time on ~6K chunks — worth checking ours doesn't share that bottleneck.